

C# Day 07 - Assignment

Part 1: Class Definitions

You are required to create three distinct classes: `HireDate`, `Department`, and `Employee`.

General Requirements for EVERY class:

For all three classes listed below, you **must** implement the following:

1. Appropriate **Properties** (Getters/Setters) for all attributes.
2. A **Default Constructor** (parameterless constructor) that initializes properties with logical default values.
3. A **Parameterized Constructor** that takes arguments to initialize all properties.
4. An **Overridden `ToString()`** method that returns a well-formatted string containing the object's data.

Class 1: `HireDate`

- **Attributes:** `Day` (int), `Month` (int), `Year` (int).

Class 2: `Department`

- **Attributes:** `DeptId` (int), `DeptName` (string).

Class 3: `Employee`

- **Attributes:** `Id` (int), `Name` (string), `Age` (int), `Salary` (double), `HireDate` (of type `HireDate`), `Department` (of type `Department`).

Part 2: Main Program Execution

Inside your `Main` method, write the code to execute and test the following scenarios:

Task A: Single Employee Data Handling

1. Instantiate a single `Employee` object.
2. Read and assign values to all of its properties (including initializing its inner `HireDate` and `Department` objects).
3. Write (print) the employee's data to the console using the overridden `ToString()` method.

Task B: Array Sorting

1. Create an array of `10 Employee` objects.
2. Populate the array with hardcoded or randomly generated data.
3. Print the array elements to the console.
4. **Sort** the array of employees based on the **Month of their Hire Date** in ascending order. **You must implement the `IComparable` interface in the `Employee` class and use the `CompareTo` method to perform the sorting.**
5. Print the array again after sorting to demonstrate that your logic works correctly.

Part 3: Advanced Concepts (Try It Yourself)

To demonstrate a deeper understanding of OOP and C# architecture, complete the following two challenges:

Challenge 1: The Singleton Pattern

- Create a new class (e.g., `CompanySystem` or `Configuration`) and apply the **Singleton Design Pattern** to it.
- Write code in the `Main` method to prove that multiple calls to retrieve the class instance will always return the exact same object in memory.

Challenge 2: Custom Object Equality

- Create a `Point` class with two integer properties: `X` and `Y`.
- By default, comparing two different objects checks for reference equality. **Override the `Equals(object obj)` method** in your `Point` class so that two distinct `Point` objects are considered equal if they have the exact same `X` and `Y` coordinate values.
- Test your implementation in the `Main` method by comparing two identical points and printing the result.